جامعة الإمام عبدالرحمن بن فيصل
IMAM ABDULRAHMAN BIN FAISAL UNIVERSITY
كلية علوم الحاسب وتقنية المعلومات
College of Computer Science and Information Technology

CSC 305: Software Engineering

Term: 1

Academic Year: 2025/2026

Course Coordinator: Dr. Rahma Ahmed

FitHub

Smart Gym & Wellness Management System

FITHUB

Supervisor: Ms. Maha Asiri

List Of Tables

List Of Figures

1. Introduction

This chapter presents the content and structure of the Software Requirements Specification (SRS) for FitHub – a smart gym and health management system. It includes its purpose, target audience, scope, rules, and references.

1.1 Purpose

This document aims to provide a comprehensive and complete description of the requirements for implementing FitHub – a smart gym and health management system. It defines the system's functional and non-functional requirements, details the external interfaces, limitations, and performance expectations. This document serves as the official reference for design, implementation, testing, and acceptance.

Target Audience:

- Project Supervisor: To review the specifications and provide guidance for necessary improvements.
- Customers/ Stakeholders: To verify the completeness, accuracy, and alignment of requirements with project objectives.
- Project Team: To use as an official reference for design, implementation tasks, test planning, and verification.

1.2 Document Conventions

- Modeling: UML (Use Case, Activity) and ERD, PK/FK/UK labeled on diagrams.
- Requirement wording: “The system shall …” with priority tags MUST / SHOULD / MAY; IDs as FR-X.Y (functional) and NFR-Area-# (non-functional).
- Naming: Entities in PascalCase (Member, Trainer, Session); database columns in snake_case (member_id, session_name).
- Data formats: Date YYYY-MM-DD, 24-hour time; default time zone Asia/Riyadh (UTC+3); duration in minutes, calories in kcal.
- Status values: Session {Scheduled, Completed, Cancelled}, Attendance {Present, Absent, Late}, Account {Active, Inactive}.
- Integrity rules: Unique email and phone_number per user; unique (member_id, session_id) in Attendance.
- Figures: All diagrams labeled Figure X.Y and referenced in the text; ERD and UML diagrams included in Appendices.

1.3 Intended Audience and Reading Suggestions

Target Audience:

- Client/ Instructor: Verifies project scope and alignment with objectives.
- Developers (BE/FE): Design and implement features and the application interface.
- Database Designers: Design the schema, constraints, and data integrity rules.
- Testers/QA: Plan test cases and ensure acceptance criteria are met.
- UI/UX Designers: Design user flows, screens, and error behavior.
- Security/ DevOps (if applicable): Enhance, deploy, and monitor.
- Members/ Trainers(stakeholders): Track project scope, risks, and milestones.

Reading suggestions (what matters most to each role)

Reading order (recommended):

Introduction → Overall Description → Specific Requirements (FR + Use Cases) → External Interfaces → Non-Functional → Appendices (ERD, UML, Data Dictionary).

Table 1 Intended Audience and Reading Suggestions

Role	Focus Section
Client / Instructor	1. Introduction, 2. Overall Description, 3 (overview), 5. Non-Functional, 6. Appendices (diagrams)
Developers (BE/FE)	2. Overall Description, 3. Specific Requirements (FR/Use Cases), 4. External Interfaces, 6. Appendices (ERD, UML)
Database Designer	2.5 Constraints, integrity in 5.7 Data Quality, Appendices (ERD, Data Dictionary)
Testers / QA	3. Specific Requirements (FR & Use Case Specifications), 5. Non-Functional, traceability in Appendices
UI/UX Designers	3.2 Use Case Specs, 4.1 User Interfaces, activity flows in Appendices
Security / DevOps	5.2 Security, 4.4 Communication Interfaces, availability/backups in 5.3/5.9
Members / Trainers (stakeholders)	1 & 2 summaries; feature list in 3.1

1.4 Scope of the System

This document covers all the requirements for the FitHub web application. The application will provide end users with a platform to discover and book training sessions, log workouts, and track attendance and achievements, while enabling trainers to create and manage sessions and view reports.

Key Features:

- Accounts & security: Register, Login, Logout, Forgot Password, Role Based Access Control (Member/ Trainer).
- Sessions: Member browse/ filter/ book. Trainer creates/updates/cancel and view rosters.
- Attendance: Mark per session (Present/ Absent/ Late), member history.
- Workouts: Log activity, duration, calories, date, view history/streak.
- Achievements: Auto-award badges (e.g., streaks/ totals), achievements view.
- Dashboards & reports: Member summary, Trainer period reports (weekly/monthly/yearly).

Out of Scope (current release):

Native mobile apps, payments/billing, nutrition plans, wearable auto-sync, multi-branch admin, push/ mobile notifications, in-app chat, and payroll/ compensation.

1.5 References

- **IEEE Std 830-1998.** IEEE Recommended Practice for Software Requirements Specifications. IEEE Computer Society, 1998.
- **Sommerville, I.** (2015). Software Engineering (10th ed.). Pearson Education.
- **CSC 305**: Software Engineering Course Materials, Term 1, AY 2025/2026 — Prepared by Dr. Rahma Ahmed (slides & rubric).

2. Overall Description

2.1 Product Perspective

FitHub is a standalone web-based application designed as an independent software product for gym and wellness management. It does not connect with external systems such as payment gateways, wearable devices, or third-party fitness apps. The system operates self-contained within a web browser environment, with all data processing, storage, and user interactions handled internally via a backend server and database.

Figure 1 shows the system Architecture

2.2 Product Functions

FitHub provides a comprehensive platform for managing gym operations and user engagement. The major high-level functions include:

- **User Authentication**: Login for members and trainers
- **Workout Logging**: Members can log workout types, duration, and calories burned
- **Session Booking**: Members can book upcoming training sessions; trainers can create and manage sessions
- **Progress Tracking**: Weekly and monthly analytics with charts for workouts and calories
- **Gamification**: Achievement badges for streaks, workout milestones, and goals
- **Trainer Dashboard**: Overview of active members, session stats, and recent activity
- **Member Management**: Trainers can view member profiles, attendance, and engagement
- **Performance Reports**: Trainers can view session analytics and attendance breakdowns

2.3 User Classes and Characteristics

FitHub supports two primary user classes, each with distinct roles and access levels. Users are assumed to have basic familiarity with web interfaces, but no advanced technical skills are required.

Table 2 User Classes and Characteristics

User Class	Description	Skills and knowledge
Gym member	The Member is a fitness-focused user who registers, logs workouts, tracks streaks, and manages session bookings through a personalized dashboard.	Basic proficiency in web navigation and a willingness to consistently log personal fitness data.
Gym Trainer	The Trainer is an authorized gym professional who manages class scheduling, supervises member activity, and monitors overall performance through dedicated dashboard and reporting tools.	Moderate proficiency in web administration and knowledge of the fitness plans and session types being managed.

2.4 Operating Environment

FitHub is a web-based platform accessible through modern browsers on various devices. The system is designed to support both members and trainers across the following environments:

Hardware Platforms:

- Standard personal computers (PCs) or laptops with internet connectivity. No specialized hardware is required for end-users.

Software Platforms:

- Web browsers (Chrome, Safari, Firefox, Edge)

No installation is required; users access FitHub directly through the website. All features - including workout logging, session booking, progress tracking, and dashboard analytics - are fully functional across supported devices.

2.5 Design and Implementation Constraints

- **Programming Language and Tools**: Frontend developed using HTML5, CSS3, and JavaScript; backend implemented in Python with the Flask framework. Database managed via SQLite for simplicity.
- **Development Tools**: Integrated Development Environment (IDE) such as Visual Studio Code; version control via GitHub; API testing with Postman; UI prototyping with Figma.
- **Project Timeline**: Must be completed within the CSC 305 course semester (approximately 15 weeks), with milestones for planning, design, implementation, and testing.
- **Resource Limitations**: Use of open-source, free tools only; no commercial licenses or paid services.
- **Performance and Quality**: Code must be modular and maintainable; system must handle basic load without degradation

2.6 Assumptions and Dependencies

Assumptions

- Users (members and trainers) have access to a stable internet connection and a compatible web browser.
- Members and trainers are registered through the platform and have valid login credentials.
- Trainers are responsible for creating and managing training sessions.
- Members are expected to log workouts and monitor their progress regularly

Dependencies

- Email functionality is required for login and account-related communication.
- Availability of development tools like VS Code, GitHub and libraries like Flask.
- Calendar-based session scheduling depends on accurate date and time inputs from trainers.
- The system relies on internal data structures to calculate progress analytics, attendance rates, and achievement milestones.

3. Specific Requirements

3.1 Functional Requirements

3.1.1 Common functionalities

3.1.1.1 Landing page

The landing page is the public entry point for all users (Members, Trainers). It communicates the product's value and provides quick access to authentication.

The system shall:

- Provide a branded landing page with logo, product headline, brief value proposition, and primary CTAs for Login and Register.
- Present role-agnostic entry (same login/ register for all; role is enforced after authentication).
- Load critical assets efficiently and serve all resources over HTTPS.

3.1.1.2 Authentication-Login

The system shall:

- Provide a unified Login form with Email and Password fields.
- Validate inputs on client and server (email format, non-empty password).
- Authenticate the user and issue a JWT access token on success.
- Redirect the user to the appropriate dashboard based on role (Member →Member Dashboard, Trainer→ Trainer Dashboard).
- Return standardized errors for invalid credentials (401) or locked/ suspended accounts (403, if enabled).
- Apply rate limiting for repeated failed attempts and support optional account lockout (configurable).

3.1.1.3 Authentication- Register

The system shall:

- Provide a Register form with Full name, Email, and Password.
- Enforce password policy (Minimum length, complexity).
- Ensure Email uniqueness; return 409 conflicts if email already exists.
- Create the user with appropriate role.
- Show success message and route the user to Login.

3.1.1.4 Authentication — Forgot Password

The system shall:

- Allow users to request a password reset via email with a time-limited token.
- Allow users to set a new password using a valid, unexpired reset token.

Table 3 Authentication Use Case Specifications (Login & Register& forgot password)

Use Case	Actors	Preconditions	Main Flow	Alternative Flows	Postconditions
UC-A1: Login	User (Member/ Trainer)	Registered account valid credentials	1) Open Login 2) Enter Email & Password 3) System validates 4) System issues JWT 5) Redirect by role	Invalid credentials (401) Account locked (if enabled 403) Network/server error retry guidance	User authenticated Token/ session active Role based redirect completed
UC-A2: Register	Visitor	Email not used Required fields provided	1) Open Register 2) Enter Full name, Email, Password 3) System validates & checks uniqueness 4) System creates user with role 5) Show success 6) Route to Login	Email exists (409) Validation errors (400) with inline messages	New account created and ready to login
UC-A3: Request Password Reset	User (Member/ Trainer)	No active session	1) Open Forgot Password 2) Enter Email 3) System validates & generates time-limited token 4) System sends reset link to email	Email not registered→ return generic success (no enumeration)	Rest email sent if account exits; request accepted
UC-A4: Confirm Password Reset	User (Member/ Trainer)	Valid	1) Open Reset Link 2) Enter New Password 3) Save 4) System validates & updates password; invalidates token	Invalid/ expired token→ 400	Password updated; user can log in

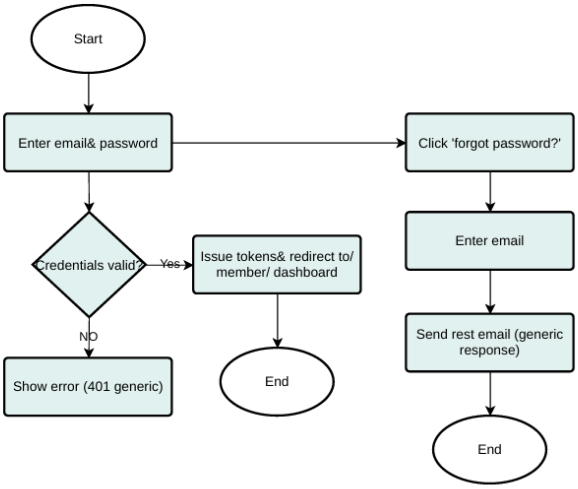

Figure 2 Member Login& Forgot Password

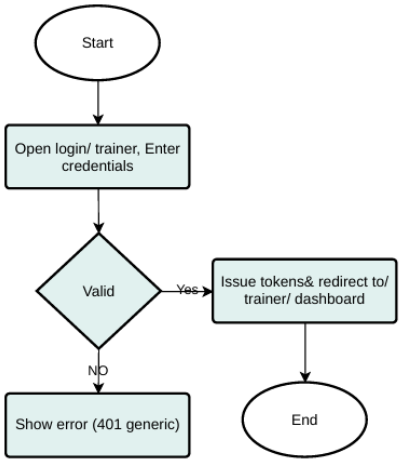

Figure 3 Trainer Login

3.1.2 Members Functionalities

3.1.2.1 Member Dashboard

A personalized overview that summarizes key performance indicators and recent activity for the logged-in member.

The system shall:

- Display KPIs: Current Streak, Total Workouts, This Month Sessions, Average Duration, Total Calories.
- List Recent Workouts with quick access to details.
- Highlight Upcoming Sessions with quick actions (Cancel/Details).
- Include a compact Weekly Activity widget (workouts per day) to visualize trends
- Gracefully handle no-data states with helpful guidance (e.g., “Log your first workout”).

Table 4 Member Dashboard Use Case Specification

Use Case	Actors	Preconditions	Main Flow	Alternative Flows	Postconditions
UC-MD1: View Member Dashboard	Member	Member authenticated	1) Open Member Dashboard 2) System aggregates KPIs, recent workouts, and upcoming sessions 3) Display	No data → Empty states for widgets	Member sees a consolidated snapshot and can navigate to details

3.1.2.2 Workouts

Members can log workouts, review history and see summary stats that feed streaks and analytics.

The system shall:

- Allow Members to log a workout with: type (cardio, strength, etc.), date, duration (min), calories, notes, and optional exercises.
- Validate inputs (non- empty type/date, positive duration/ calories).
- Show workout history and summary statistics.
- The system updates aggregated metrics (weekly/monthly sets and streak) after each creation, update, or deletion.
- The system maintains ownership rights, ensuring users can only access their own workouts.

Table 5 Workout Use Case Specifications

Use Case	Actors	Preconditions	Main Flow	Alternative Flows	Postconditions
UC-W1: Log workout	Member	Member authenticated	1) Open Log Workout 2) Fill fields 3) Submit 4) System validates & saves 5) Update totals/streak	Validation error → 400 with inline messages	Workout created metrics updated
UC-W2: View Workout History	Member	Member authenticated	1) Open My Workouts 2) System lists workouts + summary totals	No records → Empty state with guidance	Member reviews history & stats

3.1.2.1.1 Sessions (Booking)

Members can discover available sessions and manage bookings.

The system shall:

- List Available Sessions with title, trainer, date/time, duration, capacity, location, and current fill.
- Let Members book a session if capacity allows and no time conflict exists.
- Let Members cancel a booking subject to a configurable cutoff.
- Enforce capacity control and no double-booking.
- Show My Upcoming Sessions with quick actions (Cancel, View details).

Table 6 Session Booking Use Case Specifications (Member)

Use Case	Actors	Preconditions	Main Flow	Alternative Flows	Postconditions
UC-S2: Book Session Member	Member	Member authenticated session exists seats available	1) Open-Available Sessions 2) Click Book 3) System checks capacity/conflicts 4) System creates booking	Full → reject Conflict → reject Validation error → 400	Booking created capacity decremented
UC-S3: Cancel Booking Member	Member	Member authenticated active booking	1) Open My Upcoming Sessions 2) Click Cancel 3) System checks cutoff 4) Remove booking	After cutoff → reject per policy	Booking removed capacity increased

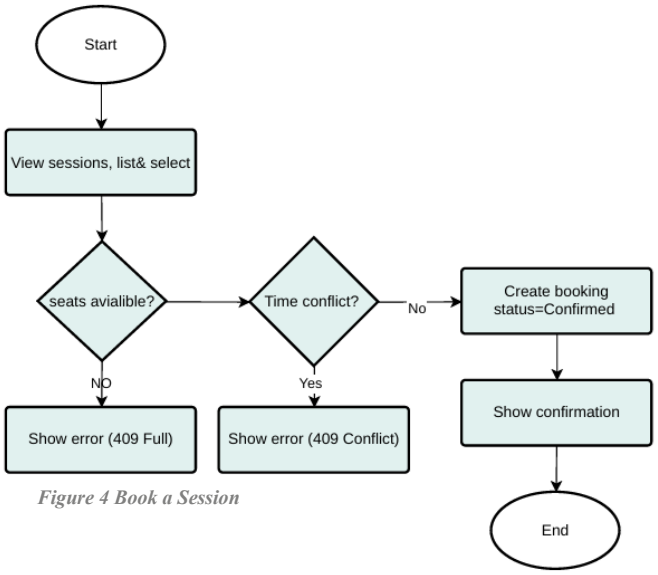

Figure 4 Book a Session

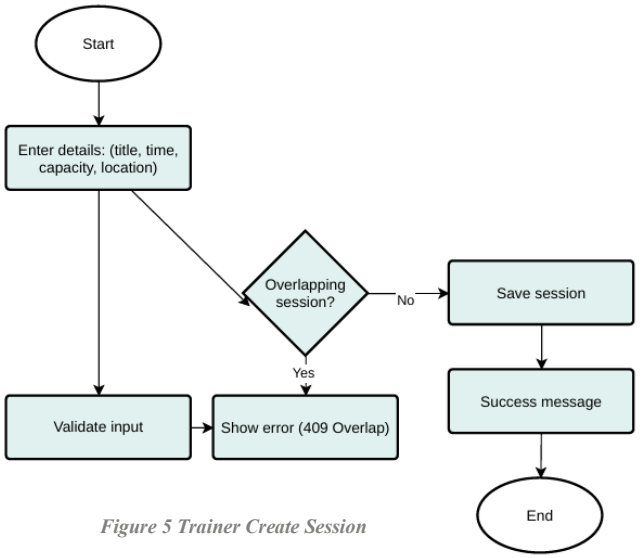

Figure 5 Trainer Create Session

3.1.2.2 Progress & Analytics

Visualize performance trends (weekly/monthly/overall), streaks, and activity breakdowns.

The system shall:

- Compute and display KPIs: total workouts, total calories, average time, current streak.
- Show Weekly Activity charts with filter by range.
- Handle no-data scenarios with helpful guidance.
- Keep analytics consistent with the workout store.

Table 7 Member Progress & Analytics Use Case Specification

Use Case	Actors	Preconditions	Main Flow	Alternative Flows	Postconditions
UC-P1: View Progress & Analytics Member	Member	Member authenticated	1) Open Progress 2) System aggregates KPIs & weekly chart 3) Display	No data → Empty states	Member understands trends & progress

3.1.2.3 Member Profile

Members view and update their own profile.

The system shall:

- Show profile fields: Full Name, Email, Phone, Member Since, Achievements (if any).
- Allow editing allowed fields (e.g., Phone, Avatar), with validation.
- Return 404 if profile not found; 401 if not authenticated.

Table 8 Member Profile Use Case Specifications

Use Case	Actors	Preconditions	Main Flow	Alternative Flows	Postconditions
UC-M3: View My Member Profile	Member	Member authenticated	1) Open My Profile 2) System returns profile (with linked user info)	Not found → 404	Profile visible
UC-M4: Update My Member Profile	Member	Member authenticated	1) Open My Profile 2) Edit fields 3) Update fields 4) Save 5) System validates & updates	Validation error → 400	Profile updated

3.1.3 Trainer Functionalities

3.1.3.1 Trainer Dashboard

A high-level snapshot for trainers.

The system shall:

- Show KPIs: Active Members, Sessions This Week, Avg Attendance, Top Performer.
- List Today's Sessions and Recent Activity.

Degrade gracefully if data is unavailable (placeholders/empty states).

Table 9 Trainer Dashboard Use Case Specification

Use Case	Actors	Preconditions	Main Flow	Alternative Flows	Postconditions
UC-TD1: View Trainer Dashboard	Trainer	Trainer authenticated	1) Open Dashboard 2) System fetches KPIs + lists	Missing data → placeholders	Trainer sees status & activity

3.1.3.2 Member Management

Trainers manage member records.

The system shall:

- List members with key fields (status, email, phone, joined, last activity, attendance%).
- Create a member profile (link to existing user or create both, per policy).
- Provide a member's detail view (read-only KPIs, recent activity).

Table 10 Member Management (Trainer) Use Case Specifications

Use Case	Actors	Preconditions	Main Flow	Alternative Flows	Postconditions
UC-M1: List Members	Trainer	Trainer authenticated	1) Open Members 2) System lists with key fields	Unauthorized → 401 non-trainer → 403	Members displayed
UC-M2: Create Member	Trainer	Trainer authenticated valid payload	1) Click Add Member 2) Submit 3) System validates & creates	Duplicate user_id → 409 Validation error → 400	Member created
UC-M5: View Member Details Trainer	Trainer	Trainer authenticated member exists	1) Click View Details 2) System shows profile + stats	Not found → 404	Detail visible

3.1.3.3 Session Management

Trainers create and update sessions.

The system shall:

- Create a session with title, datetime, duration, capacity, location.
- Update existing sessions; enforce future-date and capacity ≥ 1 rules.
- Show fill rate and booked members list.

Table 11 Session Management (Trainer) Use Case Specifications

Use Case	Actors	Preconditions	Main Flow	Alternative Flows	Postconditions
UC-S1: Create Session	Trainer	Trainer authenticated valid data	1) Open Create Session 2) Fill fields 3) Save 4) System validates & creates	Validation → 400	Session created
UC-S4: Edit Session	Trainer	Trainer authenticated session exists	1) Open session 2) Modify 3) Save	Not found → 404	Session updated

			4) System validates & updates		

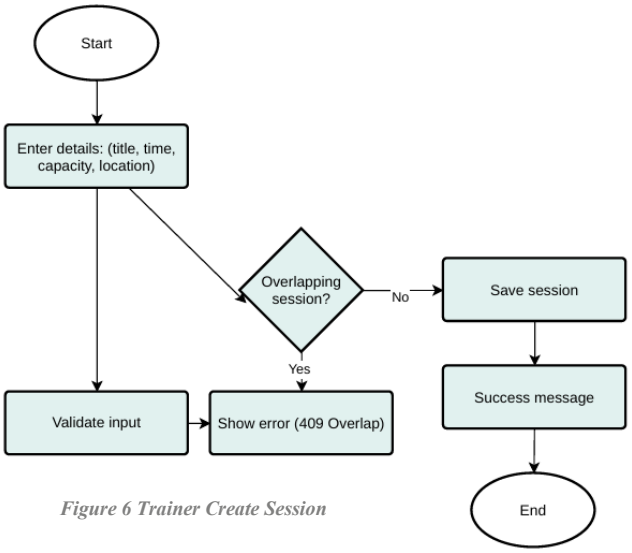

Figure 6 Trainer Create Session

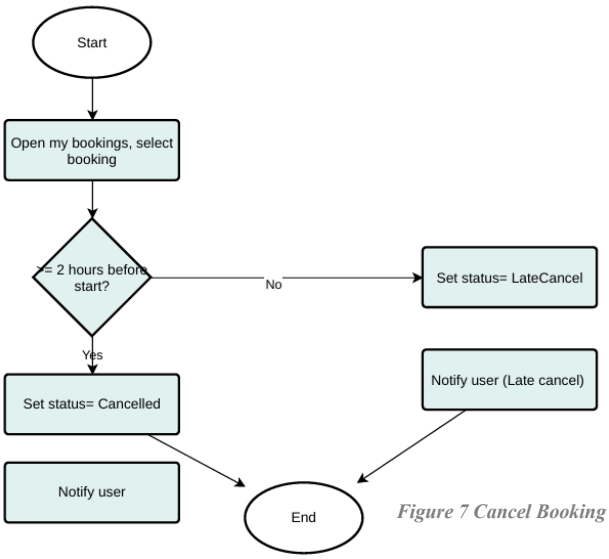

Figure 7 Cancel Booking

3.1.3.4 Reports

Performance analytics for trainers.

The system shall:

- Compute weekly/monthly KPIs: sessions, total attendance, average per session, fill rate.
- Show daily breakdown charts with filters and export (CSV/PDF) if required.
- Ensure numbers match the bookings/sessions store.

Table 12 Trainer Reports Use Case Specification

Use Case	Actors	Preconditions	Main Flow	Alternative Flows	Postconditions
UC-TR1: View Performance Reports	Trainer	Trainer authenticated	1) Open Reports 2) Select range 3) System aggregates KPIs & charts	No data → Empty state	Trainer reviews analytics

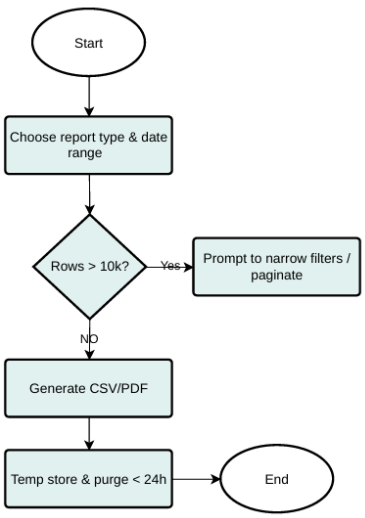

Figure 8 Reports Export

3.1.3.5 Trainer Profile

Trainers view their own profile and career info.

The system shall:

- Show profile fields: Full Name, Email, Phone, Specialization, Certification, Trainer Since, Performance Overview.
- Trainer can also edit and update personal information such as phone number, email, and specialization.
- Restrict editing (if any) per policy; otherwise read-only.

Table 13 Trainer Profile Use Case Specifications

Use Case	Actors	Preconditions	Main Flow	Alternative Flows	Postconditions
UC-T1: View My Trainer Profile	Trainer	Trainer authenticated Profile exists	1) Open Profile/Dashboard 2) System returns profile	Not found → 404	Profile visible
UC-T2: Update My Trainer Profile	Trainer	Trainer authenticated Profile exists	1) Open Profile/Dashboard 2) Edit fields 3) Update fields 2) Save 3) System validates & updates	Validation error → 400 Unauthorized → 401	Profile Updated

4. External Interface Requirements

4.1 User Interfaces

The following subsections describe the user interfaces for the **FitHub** web application. Each interface details the layout elements, data fields, and user interactions required to support system functionality for both members and trainers.

4.1.1.1 Start Page Interface

The **Start Page** is the first screen users see when accessing **FitHub**. It introduces the platform and allows visitors to choose how they wish to continue either as a **Trainer** or as a **member**.

There are two main action buttons:

- **"Get Started"** — directs members to the **Member Login Page**, where they can sign in or register if they don't have an account.
- **"Trainer Login"** — redirects trainers to the **Trainer Login Page** for secure access to their dashboard.

A simple footer with contact information and social links appears at the bottom.
Overall, the Start Page provides a welcoming and motivational entry point that clearly guides users to continue as either a trainer or a member.

4.1.1.2 Sign In Interface

The **Sign In Interface** allows both members and trainers to access their accounts using the same form.
It contains fields for **Email** and **Password**, a **Sign In** button, and links for **Forgot Password** and **Register** (for new members).

The system validates the input ensuring the email format is correct, and the password is not empty before authenticating.
After successful sign-in, the user is redirected to their respective dashboard (Member or Trainer).
Invalid credentials show an error message, and repeated failed attempts may trigger account lockout.

Table 14: Sign In Interface Fields

Field Name	Format	Level	I/O	Comment (Constraint)
Email	Text	Required	I	Must be a valid email format
Password	Encrypted Text	Required	I	Cannot be empty, min 6 characters
Sign In Button	Button	Required	I	Submits data for authentication
Forgot Password	Link	Optional	I	Opens password recovery page
Register Now	Link	Optional	I	Redirects to registration page

4.1.1.3 Member Dashboard Interface

The **Member Dashboard** provides a summary of a member's performance and activity.
It displays key metrics such as **Current Streak**, **Total Workouts**, **Sessions This Month**, **Average Duration**, and **Total Calories**.
It also includes sections for **Recent Workouts**, **Upcoming Sessions**, **Achievements**, and a **Weekly Activity Chart showing** workout trends.

This interface helps members track their overall progress, view upcoming sessions, and stay motivated with achievement highlights.

Table 15: Member Dashboard Interface Fields

Field Name	Format	Level	I/O	Comment (Constraint)
Performance Metrics	Display	Required	O	Auto-calculated values only
Recent Workouts	List View	Required	O	Displays latest logged workouts
Upcoming Sessions	List View	Optional	O	Shows only booked sessions
Achievements	Cards	Required	O	Shows earned and locked badges
Weekly Activity Chart	Graph	Optional	O	Updates automatically with new data

4.1.1.4 Trainer Dashboard Interface

The **Trainer Dashboard** gives trainers a quick overview of their members, sessions, and performance statistics. It displays key KPIs such as **Active Members**, **Sessions This Week**, **Average Attendance**, and **Top Performer**. Additional sections include **Today's Sessions**, **Recent Activity**, and **Weekly Performance Summary**.

If data is missing or unavailable, placeholder text or empty states are shown to maintain layout consistency.

Table 16: Trainer Dashboard Interface Fields

Field Name	Format	Level	I/O	Comment (Constraint)
Trainer KPIs	Display	Required	O	Auto-generated statistics
Today's Sessions	List View	Required	O	Displays sessions scheduled for today
Recent Activity	Feed List	Required	O	Updates automatically with changes
Weekly Performance	Graph	Optional	O	Shows current weekly performance trends

4.1.1.5 Workout Interface

The **Workout Interface** allows members to log and view their workout activities.
It includes a form to record **Workout Type**, **Duration**, **Calories Burned**, **Date**, and **Notes**.
All inputs are validated to ensure that fields are filled correctly, and values are positive.

The **Workout History** section lists all previous workouts in a card-style view, showing details like type, date, exercises, duration, and calories burned.
After a workout is logged, the system updates the member's analytics and streak automatically.

Table 17: Workout Interface Fields

Field Name	Format	Level	I/O	Comment (Constraint)
Workout Type	Combo Box	Required	I	Must select one workout type
Duration (Minutes)	Number	Required	I	Must be a positive number
Calories Burned	Number	Optional	I	Must be a positive number if entered
Date	Date Picker	Required	I	Cannot be in the future
Notes	Text Area	Optional	I	Max 100 characters
Add Workout Button	Button	Required	I	Saves workout after validation
Workout History	List View	Required	O	Shows only the user's workouts

4.1.1.6 Member Management Interface

The **Member Management Interface** allows trainers to manage member records.
It shows the **total number of members** and a **table of all registered members** with fields like name, email, phone, join date, status, and attendance rate.
Trainers can **add new members**, **edit information**, and **view attendance or activity summaries**.
Each member also has a detailed view showing KPIs and recent activity.

Table 18: Member Management Interface Fields

Field Name	Format	Level	I/O	Comment (Constraint)
Total Members	Display	Required	O	Count updates automatically
Member List	Table	Required	O	Lists valid registered members only
Add Member Button	Button	Required	I	Adds a new member after validation
Member Actions	Buttons	Optional	I	Allows view or edit of existing members
Search Bar	Text	Optional	I	Must match existing member name/email

4.1.1.7 Sessions Interface

The **Sessions Interface** is used by both trainers and member
Trainers can create and manage sessions, while members can view and book them.
Each session displays its title, trainer, date/time, duration, capacity, and availability.
The system prevents overbooking, double-booking, and allows members to cancel before a cutoff time.
Trainers can also view the fill rate and booked member list.

Table 19: Sessions Interface Fields

Field Name	Format	Level	I/O	Comment (Constraint)
Session Details	Text/Display	Required	I/O	Title, date, and capacity cannot be empty
Participants	Display	Required	O	Updates automatically as members join
Book Session	Button	Member Only	I	Allowed only if session not full
Cancel Booking	Button	Member Only	I	Only before the cutoff time
Create Session	Button	Trainer Only	I	Requires valid title, date, and location
Booked Members List	Display	Trainer Only	O	Shows names of booked members
Status Indicator	Display	Optional	O	Shows “Available” or “Full” only

4.1.1.8 Reports Interface

The **Reports Interface** visualizes performance trends for members and trainers.
It includes charts showing weekly, monthly, and overall statistics like **total workouts**, **total calories**, **average duration**, and **streaks**.
Members can filter by date range, while trainers can view aggregated stats such as total sessions and attendance rates.
If no data is available, a helpful message is shown.

Table 20: Reports Interface Fields

Field Name	Format	Level	I/O	Comment (Constraint)
Date Range Selector	Date Picker	Required	I	Must select valid date range
Metric Selector	Combo Box	Optional	I	Choose from available metrics
Progress Chart	Graph	Required	O	Displays based on selected data
Summary Cards	Display	Required	O	Auto-calculated totals only
Weekly Activity Chart	Graph	Optional	O	Updates weekly
Export Button	Button	Optional	I	Exports report in PDF or CSV format

4.1.1.9 Achievements Interface

The **Achievements Interface** displays badges and milestones earned by members.
It includes unlocked badges such as 7-Day Streak and First 10 Workouts and locked ones shown in grayscale with progress bars.
This interface encourages members to maintain consistency and celebrate their progress.

Table 21 Achievements Interface Fields

Field Name	Format	Level	I/O	Comment (Constraint)
Earned Badges	Icons	Required	O	Unlocked automatically
Locked Badges	Icons (Gray)	Required	O	Locked until goals achieved
Streak Counter	Display	Required	O	Auto-updates daily
Progress Bars	Display	Optional	O	Shows progress toward next badge

4.1.1.10 Profile Interface

The **Profile Interface** allows both members and trainers to view and update personal details.
Members can update their **phone number**, **goal**, and **profile picture**, while trainers can also edit their **specialization** and **certifications**.
Both roles can view information such as **Full Name**, **Email**, and **Join Date**.
All inputs are validated before saving.

Table 22 Profile Interface Fields

Field Name	Format	Level	I/O	Comment (Constraint)
Full Name	Text	Required	I/O	Letters only, max 50 characters
Email	Text	Required	I/O	Must be valid email format
Phone Number	Text	Optional	I/O	Digits only, 10–15 numbers
Member Since / Trainer Since	Display	Required	O	Auto-generated, not editable
Fitness Goal	Combo Box	Member Only	I/O	Must select from available goals
Trainer Details	Text	Trainer Only	I/O	Optional field for specialization info
Profile Picture	File Upload	Optional	I	JPG/PNG only, max 2 MB
Save Changes	Button	Required	I	Saves after input validation
Delete Account	Button	Optional	I	Requires confirmation to proceed

4.1.1.11 Contact Interface

The **Contact Interface** allows users to send inquiries or feedback to FitHub support.
The form includes fields for **Name**, **Email**, **Subject**, and **Message**, with an optional file attachment.
After submission, the system confirms that the message was successfully sent or displays an error if it fails.

Table 23: Contact Interface Fields

Field Name	Format	Level	I/O	Comment (Constraint)
Full Name	Text	Required	I	Letters only, cannot be blank
Email	Text	Required	I	Must be valid email address
Subject	Text	Required	I	Max 50 characters
Message	Text Area	Required	I	Max 250 characters
Attachment	File Upload	Optional	I	Accepts JPG, PNG, or PDF
Submit Button	Button	Required	I	Sends message after validation
Confirmation Text	Display	Required	O	Shows success or error message

4.2 Hardware Interfaces

The **FitHub** system is a web-based application that runs on standard hardware. It can be accessed using any device such as a desktop, laptop, tablet, or smartphone with an internet connection. Both members and trainers can use the system through a modern web browser. No additional hardware components are needed to operate the system.

4.3 Software Interfaces

The system shall interact with multiple software components and services to ensure full functionality and data consistency.

1. **Database Interface**
    - The system shall connect to a relational database (SQLite) to store and manage data for members, trainers, sessions, attendance, workouts, and achievements.
    - Communication between the application and the database shall use structured SQL queries through the Flask ORM layer.
    - The database shall enforce constraints such as unique IDs, foreign-key relationships, and referential integrity among all related tables.
2. **Web Server Interface**
    - The backend shall operate on a Flask web server and communicate with the frontend via RESTful APIs.
    - Data exchange shall use JSON format to ensure interoperability.
    - The server shall handle HTTP methods (GET, POST, PUT, DELETE) and respond with standard HTTP status codes.
3. **Authentication Module**
    - The system shall include an authentication component for registration, login, logout, password recovery, and email verification.
    - User credentials shall be hashed and salted using secure algorithms (e.g., bcrypt) before storage.
    - Token-based sessions shall be used to maintain secure and consistent authentication.

4. **Third-Party APIs (Future Integration)**
    - The system shall be designed to support future integration with external fitness or health APIs (e.g., Fitbit, Apple Health) to extend functionality and data synchronization.

4.4 Communication Interfaces

Communication between system components and external users shall take place through secure, standardized protocols.

1. **Network Protocols**
    - All client–server communication shall use **HTTPS** to protect data from interception and tampering.
    - The application shall rely on RESTful HTTP requests for all data transactions between the frontend and backend.
2. **Data Format**
    - **JSON** shall be the standard data exchange format for requests and responses.
    - All responses shall include standardized keys, messages, and status codes.
3. **Email Communication**
    - The system shall use **SMTP** to send registration confirmations, password reset links, and system notifications.
    - All outgoing emails shall follow secure authentication standards.
4. **Error Reporting and Status Codes**
    - The system shall provide meaningful HTTP status codes (200, 400, 401, 404, 500) and structured error messages in JSON format.
    - Error logs shall be recorded for troubleshooting and quality assurance.
5. **Dynamic Data Updates**
    - Dashboard and analytics pages shall support asynchronous data fetching (AJAX or API polling) to update statistics without reloading the page.

5. Non-Functional Requirements

5.1 Performance Requirements

The FitHub system shall maintain consistent speed, responsiveness, and reliability under expected loads.

1. **Response Time**
    - 90% of user requests shall receive a response within **2 seconds** under normal conditions.

- Dashboard and report pages with analytical data shall load within **5 seconds**.

2. **Throughput**
    - The system shall support at least **100 concurrent users** without noticeable performance degradation.
    - Data-fetch operations for charts and reports shall handle up to **1 000 requests per hour**.
3. **Availability**
    - The system shall maintain **99% uptime** during regular operating hours.
4. **Scalability**
    - The system architecture shall allow future scaling to support more users and datasets without major code modifications.
5. **Storage Capacity**
    - The database shall efficiently store up to **10 000 user records** and **100 000 workout/session entries**, with indexing for fast retrieval.

5.2 Security Requirements

Because FitHub handles personal fitness information, strict security measures are required to ensure data confidentiality, integrity, and availability.

1. **Authentication and Access Control**
    - Users shall authenticate using email and password credentials.
    - Role-Based Access Control (RBAC) shall restrict functional access based on user roles (Member, Trainer, Admin).
    - Administrative features shall require additional authorization checks.
2. **Data Protection**
    - All sensitive data (passwords, tokens, personal information) shall be encrypted during storage and transmission.
    - Communication between client and server shall be secured via HTTPS with SSL/TLS certificates.
3. **Session Management**
    - User sessions shall expire after **15 minutes of inactivity** to prevent unauthorized access.
    - Session tokens shall be unique and invalidated upon logout or after timeout.
4. **Backup and Recovery**
    - Automated weekly database backups shall be performed and stored in a secure, encrypted environment.
    - Recovery procedures shall ensure that no more than one day of data is lost in case of failure.

5. **Audit and Logging**
    - All critical operations (login, logout, data update, deletion, and admin actions) shall be logged with timestamps and user IDs.
    - Logs shall be reviewed periodically to detect suspicious activity or security violations.
6. **Privacy and User Consent**
    - Users shall be informed about data usage policies during registration.
    - Personal data shall not be shared with third parties without explicit consent.

5.3 Reliability and Availability

With its uptime, backup, and fault tolerance controls, the FitHub fitness system ensures a consistent user experience by guaranteeing continuous availability, safe data preservation, and smooth recovery from unexpected failures.

1. **Uptime**
   The system must maintain a minimum uptime of 99.5% during operating hours (Users anticipate that the fitness app will be accessible whenever they wish to log workouts, monitor progress, or review plans — including during weekends and early mornings).
2. **Backup**
   The system is required to conduct daily automatic backups of data and store them safely (Fitness information must be safeguarded against any loss).
3. **Fault Tolerance**
   The system should remain operational at a diminished capacity in case of hardware, network, or service disruptions (If one part fails, other components should still function without causing a complete system crash).

5.4 Maintainability

FitHub's structure is adaptable and extensible, allowing for smooth updates, improvements, and extensions with new features and upgrades without interfering with ongoing operations.

Updates and bug fixes must be able to be deployed without causing downtime (or with minimal downtime of less than 10 minutes). The system should accommodate the addition of new modules with little modification to the existing code.

5.5 Portability

The system is a web-based application that works on modern browsers such as Google Chrome, Microsoft Edge, Firefox, and Safari.
It supports both desktops (Windows, macOS, Linux) and mobile devices (Android, iOS).
No installation is required — users can access it directly through the browser.

6. Appendices

Glossary of terms

Table 24 Glossary of terms

Term	Definition
API	Application Programming Interface is a set of rules that allow different software programs to communicate with each other.
Backup	a copy of data created for recovery in case the original is lost or corrupted.
CTA	CTA stands for Call to Action, which is a prompt—like a button or a phrase—that encourages a user to perform a specific task within a software application or on a website.
Fault tolerance	a software engineering concept where a system is designed to continue operating correctly, without interruption, even when one or more of its components fail.
JWT access token	The JSON Web Token (JWT) access token is a specific type of JSON Web Token used in authentication and authorization flows, particularly within the OAuth 2.0 framework. It represents the authorization granted to a client to access specific resources on behalf of a user.
KPI	Key Performance Indicators are quantifiable measurements that help an organization, team, or individual track progress toward specific business objectives.
ORM	(Object-Relational Mapping) in the context of Flask is a software component that allows developers to interact with a relational database using Python objects and classes, instead of writing raw SQL queries.
Response time	The amount of time it takes from when a request was submitted until the first response is produced.
SQL	Structured Query Language is a domain-specific programming language designed for managing and manipulating data in relational databases.
Throughput	number of processes that complete their execution per time unit.
Uptime	time during which a machine, especially a computer, is in operation.

ERD Diagram

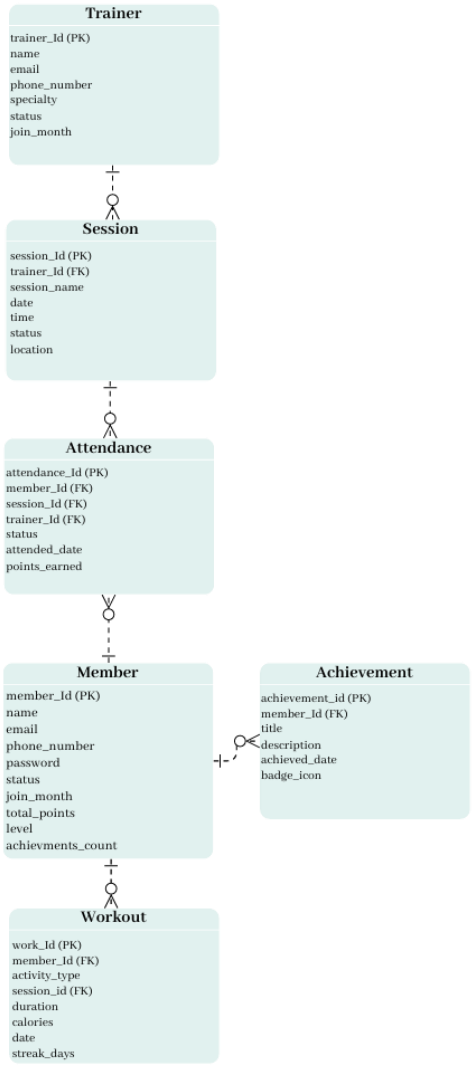

Figure 9 ERD Diagram

Use Case Diagram

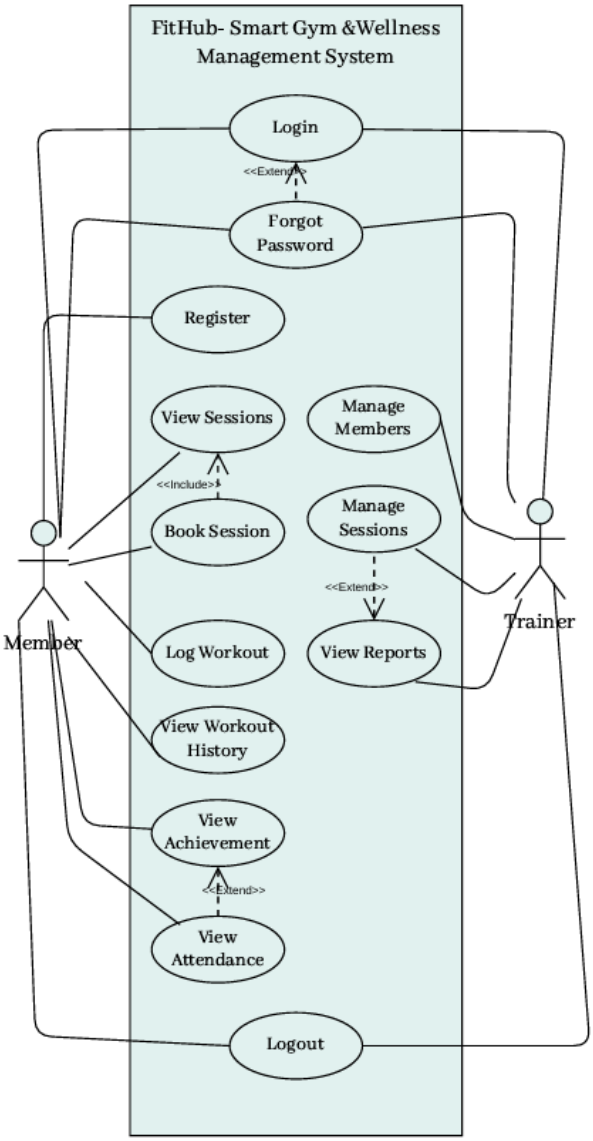

Figure : Use Case Diagram

Activity diagram

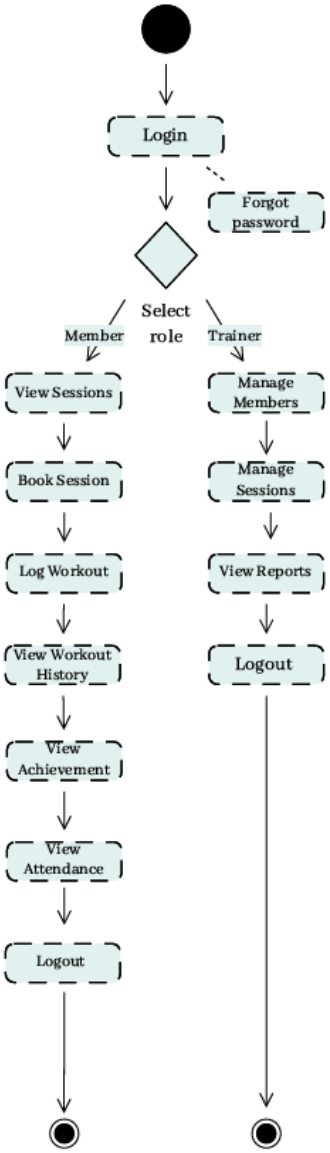

Figure 11 Activity Diagram

References to Detailed Design Documents

This Software Requirements Specification (SRS) includes key design representations, such as the Entity Relationship Diagram (Figure 9), Use Case Diagram (Figure 10), and UML Activity Diagram (Figure 11). These diagrams provide a clear view of the system's structure, interactions, and workflows. Detailed design specifications, including component-level

architecture and interface criteria, will be documented
separately in the Project Design Document (SDS) during the detailed design phase.